

STM32 NUCLEO-G031K8

Protection Controller

From Standalone STM32CubeMX Configuration to STM32CubeIDE
Build and Flash

Exact project workflow for protection_controller

Target: NUCLEO-G031K8 / STM32G031K8T6
Power Amplifier Speaker Protection Controller
25 September 2026

PHASE A - STM32CubeMX

Select the MCU, assign pins, configure ADC/GPIO, set the 64 MHz clock, choose STM32CubeIDE as the generator, then generate the project.

PHASE B - STM32CubeIDE

Open the generated project, add the protection firmware, build it, then flash the Nucleo through the onboard ST-LINK USB interface.

1. What we actually did

The important point is that we did not begin by writing firmware inside STM32CubeIDE. We first built the microcontroller hardware configuration in the standalone STM32CubeMX application and saved it as `protection_controller.ioc`. CubeMX defined the pins, ADC, GPIO behaviour, clocks, generated HAL initialization code, and created an STM32CubeIDE-compatible project. Only after that generation step did we move to CubeIDE to add the protection logic, compile, debug, and program the board.

Core principle: The `.ioc` file is the hardware configuration source of truth. If a pin or peripheral configuration must change, change it in CubeMX and regenerate; do not manually fight the generated initialization code in CubeIDE.

1.1 Final target

Item	Final project value
Development board	ST NUCLEO-G031K8
Microcontroller	STM32G031K8T6 (CubeMX project header may show STM32G031K8Tx)
CubeMX project name	<code>protection_controller</code>
Project location	<code>C:\STProjects</code>
Generated toolchain	STM32CubeIDE
Firmware package shown in saved project	STM32Cube_FW_G0 V1.6.3
CPU clock	64 MHz
Programming interface	Onboard ST-LINK over USB, SWD
Flash image	<code>Debug/protection_controller.elf</code>

1.2 End-to-end workflow

1. Open standalone STM32CubeMX and create a new MCU project for STM32G031K8T6 / STM32G031K8Tx.
2. Configure the four ADC inputs, two buttons, two relay outputs, and status LED outputs.
3. Configure the system clock to 64 MHz using HSI16 and the PLL.
4. In Project Manager, select STM32CubeIDE as the toolchain and generate code.
5. Open the generated `protection_controller` project in STM32CubeIDE.
6. Add the custom protection `.c` and `.h` modules and connect them to `main.c` inside USER CODE regions.
7. Build the project and confirm a clean compile.
8. Connect the NUCLEO-G031K8 by USB to the onboard ST-LINK, then Run/Debug the STM32 C/C++ application.
9. Confirm that programming and verification succeed, then reset/power-cycle to run the firmware from internal Flash.

2. Phase A - Configure the MCU in standalone STM32CubeMX

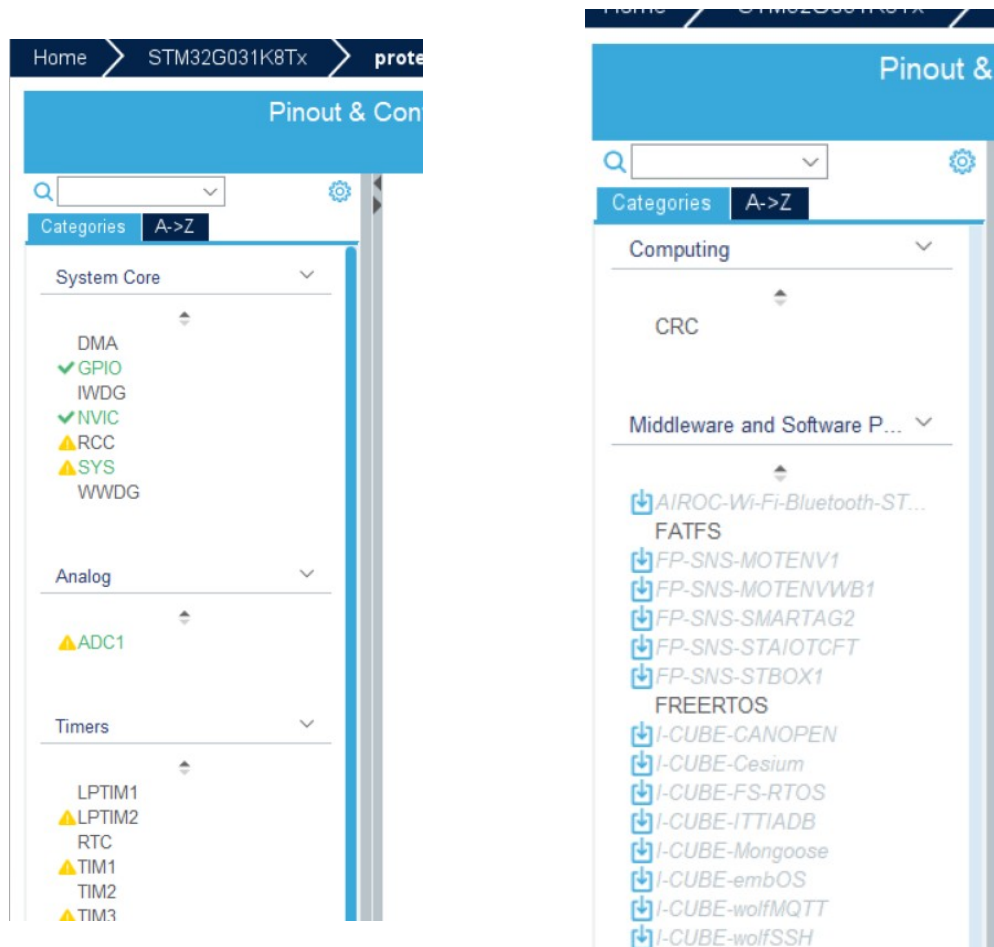
This section reproduces the part that happened before CubeIDE. This is the step that turns a blank STM32 target into a generated project with the correct clocks, ADC channels, GPIO directions, pull resistors, startup states, and HAL initialization code.

2.1 Start a new MCU project - use the MCU selector

1. Launch STM32CubeMX as a standalone application.
2. Choose New Project, then MCU/MPU Selector.
3. Search for STM32G031K8T6. CubeMX may identify the project device generically as STM32G031K8Tx; this matches the saved protection_controller.ioc shown in the screenshots.
4. Create the project. Do not start by writing a bare main.c in CubeIDE; the purpose of CubeMX is to create the hardware initialization first.
5. Save the project as protection_controller.ioc.

2.2 Pinout & Configuration page

The saved project screenshots show the standalone CubeMX Pinout & Configuration workspace. GPIO and ADC1 are the important peripherals for this controller. We did not need an RTOS, networking stack, USB middleware, or other middleware package for the protection logic.



GPIO and ADC1 are the relevant hardware categories.

No middleware or RTOS is required for this project.

2.3 Final pin assignment

The following mapping is the final configuration we settled on. It includes the later corrections: the right relay is on PA9, PB9 is the TEMP2 failure LED, and PA15/D2 is intentionally not used.

MCU pin	CubeMX function	Project signal	Direction / startup
PA0	ADC1_IN0	TEMP1	Analog input
PA1	ADC1_IN1	TEMP2	Analog input
PA4	ADC1_IN4	DC_LEFT	Analog input
PA5	ADC1_IN5	DC_RIGHT	Analog input
PA8	GPIO_Output	RELAY_L_CTRL	Output, initial LOW
PA9	GPIO_Output	RELAY_R_CTRL	Output, initial LOW
PB8	GPIO_Output	TEMP1_FAIL_LED	Output, initial LOW
PB9	GPIO_Output	TEMP2_FAIL_LED	Output, initial LOW
PA11	GPIO_Output	DC_FAULT_LED	Output, initial LOW
PA12	GPIO_Output	WAIT_LED	Output, initial LOW
PB4	GPIO_Output	OVERHEAT_LED	Output, initial LOW
PB0	GPIO_Input	MUTE_ENABLE / ENABLE_MUTE	Input with pull-up; active LOW button
PB1	GPIO_Input	FAULT_RESET	Input with pull-up; active LOW button
PA13	SYS_SWDIO	ST-LINK debug	Reserved for SWD
PA14	SYS_SWCLK	ST-LINK debug	Reserved for SWD
PA15 / D2	Unused	Do not use	Nucleo D2 pin has the board jumper-to-GND consideration

Fail-safe startup: The relay-control GPIOs were configured to start LOW. That keeps the loudspeaker relays de-energised while the MCU is resetting and before the protection firmware has decided that conditions are safe.

2.4 How to assign each GPIO in CubeMX

1. On the package drawing, click the required pin and choose the function (for example GPIO_Output, GPIO_Input, or ADC1_IN0).
2. For outputs, open System Core > GPIO and select the row for that pin. Set Output Level = Low, GPIO mode = Output Push Pull, and normally use No pull unless the external circuit requires otherwise.
3. For PB0 and PB1, choose GPIO_Input and set Pull-up. The buttons therefore read logic 1 normally and logic 0 when pressed to ground.
4. Assign a clear User Label. CubeMX uses these labels to generate pin macros in main.h. The labels should match the project signal names shown in the table.
5. Leave PA13/PA14 available for SWD by setting System Core > SYS > Debug = Serial Wire.
6. Do not assign PA15/D2 to the protection logic in this design.

2.5 ADC1 configuration

Four analogue inputs are required: two NTC temperature channels and two DC-sense channels. The final ADC setup was deliberately simple and software controlled rather than free-running.

ADC setting	Value used	Reason
Resolution	12-bit	0 to 4095 raw code range for sensor and DC measurements.

ADC setting	Value used	Reason
Data alignment	Right aligned	Natural integer ADC values for the firmware.
Channels	IN0, IN1, IN4, IN5	TEMP1, TEMP2, DC_LEFT, DC_RIGHT.
Conversion trigger	Software start	Firmware decides when each measurement occurs.
Continuous conversion	Disabled	No uncontrolled free-running conversions.
DMA continuous requests	Disabled	Measurements are handled directly by the firmware.
Scan / sequence behaviour	Single software-controlled conversions	The firmware reads the required channels as part of its sampling routine.
Common sampling time 1	160.5 ADC clock cycles	Long sampling period chosen for the divider/NTC sensing networks.
Common sampling time 2	160.5 ADC clock cycles	Same conservative sampling time for the other channel group.
ADC clock	SYSCLK source shown as 64 MHz; synchronous /2 in ADC setup	Effective ADC conversion clock about 32 MHz in the final setup.

Important: If the .ioc file is reopened later, the ADC and GPIO settings should be changed there and the project regenerated. Generated functions such as MX_ADC1_Init() and MX_GPIO_Init() should remain CubeMX-owned.

3. Clock configuration in CubeMX

The protection controller does not need an external crystal for its main CPU clock. We used the STM32 internal 16 MHz HSI oscillator and the PLL to generate a 64 MHz system clock.

3.1 Exact clock path

Clock stage	Setting	Result
HSI RC	16 MHz internal oscillator	16 MHz
PLL source	HSI	16 MHz into PLL
PLL M	/1	16 MHz
PLL N	x8	128 MHz VCO-related stage
PLL R	/2	64 MHz PLL clock
System Clock Mux	PLLCLK	SYSCLK = 64 MHz
AHB prescaler	/1	HCLK = 64 MHz
APB prescaler	/1	PCLK = 64 MHz
Cortex / SysTick / FCLK	Derived from 64 MHz HCLK	64 MHz
ADC kernel source	SYSCLK	64 MHz shown on clock tree; ADC divider applied in ADC configuration

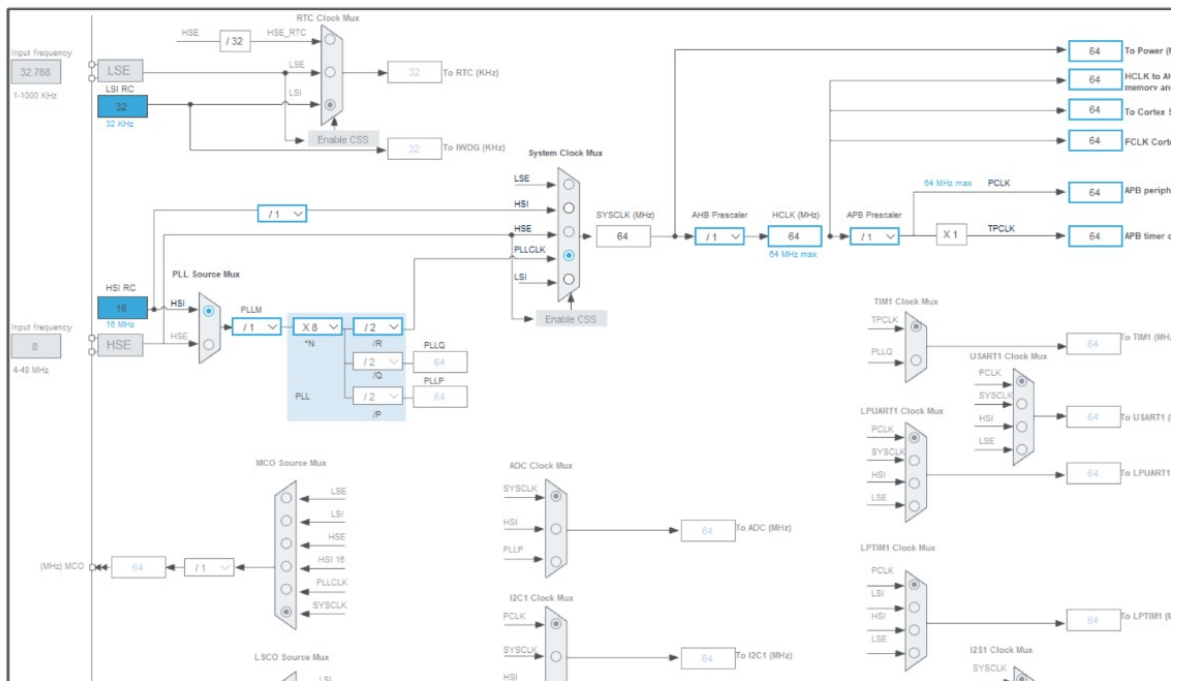


Figure 1 - Saved STM32CubeMX clock tree for protection_controller. HSI16 feeds the PLL and the system runs at 64 MHz.

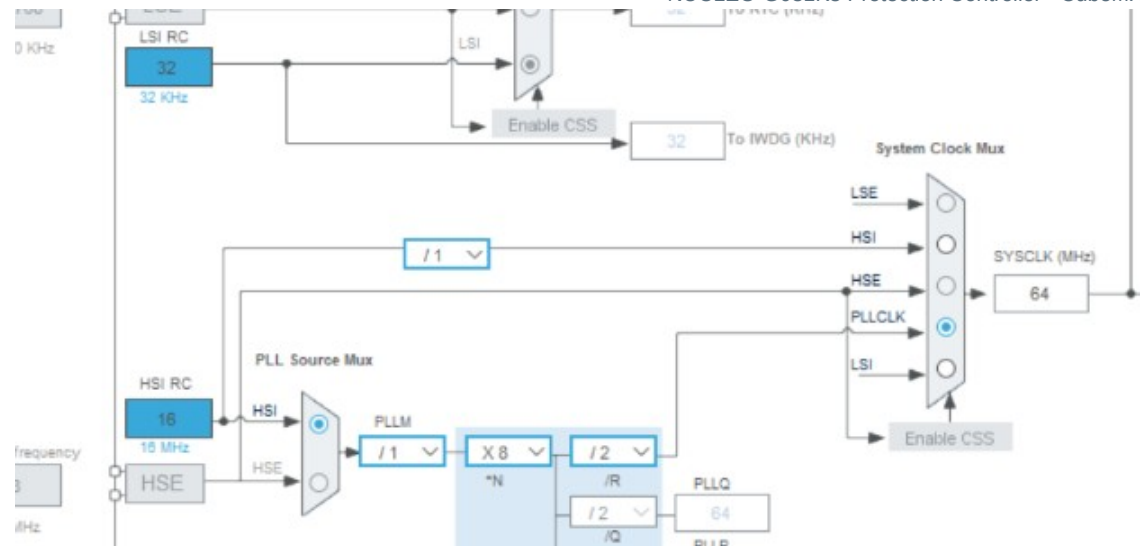


Figure 2 - PLL detail: HSI /1, x8, /2, with PLLCLK selected as SYSCLK.

3.2 What to verify before leaving Clock Configuration

- SYSCLK must read 64 MHz.
- HCLK must read 64 MHz.
- APB peripheral clock must read 64 MHz.
- The system clock source selector must be on PLLCLK.
- There should be no unresolved red clock error in the clock tree.
- Do not change the CPU frequency later in CubeIDE without updating timing-dependent assumptions; regenerate through CubeMX instead.

4. Project Manager - the step that hands the project to CubeIDE

This is the key bridge between the two applications. We configured everything in CubeMX first, then told CubeMX to generate an STM32CubeIDE project. The screenshot below is from the saved protection_controller.ioc and confirms the project-generation settings.

Figure 3 - Project Manager settings from the saved standalone CubeMX project.

4.1 Exact Project Manager values shown in the saved project

Field	Value
Project Name	protection_controller
Project Location	C:\STProjects
Application Structure	Advanced
Do not generate the main()	Not checked - CubeMX generates main.c
Toolchain Folder Location	C:\STProjects\protection_controller\
Toolchain / IDE	STM32CubeIDE
Generate Under Root	Checked
Minimum Heap Size	0x200
Minimum Stack Size	0x400

Field	Value
MCU Reference	STM32G031K8Tx
Firmware Package	STM32Cube_FW_G0 V1.6.3 in the saved screenshot
Use latest available version	Checked in the saved project
Default firmware location	Enabled

4.2 Generate the CubeIDE project

1. Save the .ioc file.
2. Click GENERATE CODE in the upper-right corner of STM32CubeMX.
3. CubeMX creates the project folder C:\STProjects\protection_controller\ together with Core, Drivers, the .ioc file, startup files, linker script, and CubeIDE project metadata.
4. When generation finishes, open the generated project in STM32CubeIDE. At this point the generated project already knows the MCU, HAL package, pin setup, ADC setup, and clock setup.
5. Keep protection_controller.ioc in the project. It is the file to reopen when hardware configuration changes are needed.

Why this matters: CubeIDE did not invent the pinout or clocks. CubeMX generated those parts first. CubeIDE was then used as the C/C++ editor, compiler, debugger, and flashing front end.

5. Phase B - Open the generated project in STM32CubeIDE

5.1 Generated project structure

After code generation, the STM32CubeIDE project contained the standard STM32 HAL structure. We then added the protection-controller modules into the generated Core folders.

```

protection_controller/
|
+-- protection_controller.ioc
+-- Core/
|   +-- Inc/
|   |   +-- main.h           [generated by CubeMX]
|   |   +-- protection.h     [our code]
|   |   +-- protection_config.h [our code]
|   |   +-- protection_io.h   [our code]
|   |   +-- protection_pins.h [our code]
|   |   +-- sensors.h        [our code]
|   +-- Src/
|   |   +-- main.c           [generated + USER CODE calls]
|   |   +-- protection.c     [our code]
|   |   +-- protection_io.c  [our code]
|   |   +-- sensors.c        [our code]
+-- Drivers/
|   +-- CMSIS/
|   +-- STM32G0xx_HAL_Driver/
+-- Debug/                    [created when building]
    +-- protection_controller.elf

```

5.2 What each custom file does

File	Purpose
protection_config.h	Timing constants, ADC thresholds, temperature trip/reset values, averaging and debounce constants.
protection_pins.h	Project-level logical pin mapping/helpers used by the protection modules.
sensors.c / sensors.h	ADC acquisition, sensor validation, temperature/DC measurement handling.
protection_io.c / .h	Relay outputs, LED outputs, button input handling.
protection.c / protection.h	Main protection state machine: startup delay, fault latch, DC persistence, thermal checks, reset rules, mute/enable behaviour.
main.c	Generated application entry point. It initializes HAL/clock/GPIO/ADC, then calls our protection module.

5.3 Preserve CubeMX USER CODE sections

The generated main.c can be regenerated later. Therefore our additions must go inside CubeMX USER CODE regions so they survive regeneration.

```

/* USER CODE BEGIN Includes */
#include "protection.h"
/* USER CODE END Includes */

int main(void)
{
    HAL_Init();

```

```

SystemClock_Config();

MX_GPIO_Init();
MX_ADC1_Init();

/* USER CODE BEGIN 2 */
Protection_Init();
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    Protection_Task();
    HAL_Delay(5);
}
/* USER CODE END 3 */
}

```

Do not move hardware initialization into our modules: SystemClock_Config(), MX_GPIO_Init(), and MX_ADC1_Init() are generated from the .ioc file. Our application code starts after those functions have configured the MCU.

5.4 Important protection constants used in the project

Constant	Value	Meaning
STARTUP_DELAY_MS	3000 ms	Relays remain disconnected for 3 seconds after startup.
SAMPLE_INTERVAL_MS	10 ms	Protection measurements are processed periodically.
BUTTON_DEBOUNCE_MS	30 ms	Debounce interval for front-panel buttons.
ADC_AVERAGE_SAMPLES	8	ADC readings are averaged to reduce noise.
TEMP_TRIP_ADC	261U	Approximate overtemperature trip point (~100 degC in the project calibration).
TEMP_RESET_ADC	342U	Approximate reset-allowed temperature (~90 degC), providing hysteresis.

5.5 Build the firmware

1. In Project Explorer, select protection_controller.
2. Use Project > Build Project, Ctrl+B, or the hammer/build icon.
3. Watch the Console and Problems views. Resolve all compile/link errors before connecting the amplifier hardware.
4. A successful Debug build creates Debug/protection_controller.elf. This ELF file contains the program that CubeIDE sends to the MCU.

6. Phase C - Flash the NUCLEO-G031K8 through onboard ST-LINK

6.1 Hardware connection for the first programming test

The NUCLEO-G031K8 contains its own ST-LINK programmer/debugger. No external ST-LINK probe and no separate USB-to-serial adapter are required.

1. Leave the Nucleo disconnected from the protection PCB for the first flash if desired. It is safe to program the complete Nucleo development board with its I/O pins otherwise unconnected.
2. Connect a USB data cable from the PC to the Nucleo onboard ST-LINK USB connector.
3. The USB connection powers the Nucleo logic and provides the SWD programming/debug connection.
4. You do not need to hold BOOT0 or connect external programming wires for this normal ST-LINK/SWD method.
5. PA13 and PA14 must remain available as SWDIO and SWCLK. This is why SYS > Debug = Serial Wire is important in CubeMX.

Board-specific caution: PA15 corresponds to the Nucleo D2 position in this project context and was deliberately left unused because of the D2-to-GND jumper arrangement. Do not assign it to a protection output without first changing/understanding the board jumper configuration.

6.2 Start the flash from CubeIDE

1. Build the project first so Debug/protection_controller.elf is up to date.
2. With the Nucleo connected by USB, choose Run > Run As > STM32 C/C++ Application. Debug > Debug As > STM32 C/C++ Application can also be used when breakpoints/variable inspection are required.
3. CubeIDE launches the ST-LINK GDB server and STM32CubeProgrammer components automatically.
4. The connection uses SWD. In the successful session the SWD frequency was 4000 kHz and the board target voltage reported about 3.25 V.
5. CubeIDE erases/programs the required internal Flash sectors, writes the ELF image to the STM32 internal Flash at the normal application base around 0x08000000, and performs verification.
6. After verification, the MCU resets and begins executing the protection firmware.

6.3 What the successful programming session looked like

```
ST-LINK GDB server
Board       : NUCLEO-G031K8
Target      : STM32G03x / STM32G04x family
Target VDD  : about 3.25 V
Interface   : SWD
SWD freq    : 4000 kHz

STM32CubeProgrammer
Flash       : 64 KB
Application : protection_controller.elf
Base        : 0x08000000
Result      : Download / programming verified successfully
```

6.4 After flashing

- The program is stored in the STM32 internal Flash; it remains there after the USB cable is removed.
- Pressing reset or powering the board again starts the flashed firmware automatically.
- The board does not need to remain attached to CubeIDE to operate.
- For standalone amplifier operation, install the programmed Nucleo on the protection PCB and power it using the intended board power arrangement.
- Before connecting valuable loudspeakers, verify ADC inputs, relay behaviour, fault LEDs, temperature sensing, and DC-fault handling with controlled tests.

7. What happens when the firmware starts

Once the MCU resets after programming, the generated HAL initialization executes first, followed by our protection state machine. The intended sequence is fail-safe: speakers begin disconnected and are only connected when the checks pass.

1. HAL and the 64 MHz system clock are initialized.
2. GPIOs are initialized. Relay control outputs begin LOW, so both speaker relays are OFF.
3. ADC1 is initialized for the temperature and DC-sense inputs.
4. Protection_Init() resets the protection state, timers, fault latches, and outputs.
5. The 3-second startup delay begins; WAIT indication is used while the speakers are intentionally disconnected.
6. Both temperature channels are checked for valid sensor readings. A missing/invalid sensor causes a fault and the relays remain OFF.
7. Left and right DC-sense inputs are monitored. A sustained dangerous DC condition raises a latched DC fault and disconnects the speakers.
8. Temperature is monitored continuously. Around the configured trip threshold (~100 degC), an overtemperature fault disconnects the relays. Reset is allowed only after cooling to around the reset threshold (~90 degC).
9. The ENABLE/MUTE button can intentionally control the connected state, while FAULT_RESET is accepted only when the monitored conditions are safe.

7.1 Fail-safe signal summary

Condition	Relay state	Indicator / action
Reset / MCU boot	OFF	Outputs start in safe LOW state.
Startup delay	OFF	WAIT indication active.
Normal, sensors valid, no DC, temperature safe	ON	Normal operation.
TEMP1 invalid	OFF	TEMP1_FAIL_LED; fault latched/handled.
TEMP2 invalid	OFF	TEMP2_FAIL_LED; fault latched/handled.
Dangerous left or right DC	OFF	DC_FAULT_LED; speakers isolated.
Overtemperature	OFF	OVERHEAT_LED; reset blocked until cooled.
Manual mute / disable	OFF	Relays intentionally de-energised.

8. Rebuild-from-zero checklist

Use this checklist if the project ever needs to be reconstructed on another PC or after deleting the generated project. It deliberately separates CubeMX work from CubeIDE work.

A. CubeMX - before opening CubeIDE

- ☐ Create MCU project for STM32G031K8T6 / STM32G031K8Tx.
- ☐ Name it protection_controller and save the .ioc.
- ☐ Set SYS Debug = Serial Wire.
- ☐ Enable PA0=ADC1_IN0, PA1=ADC1_IN1, PA4=ADC1_IN4, PA5=ADC1_IN5.
- ☐ ADC: 12-bit, right aligned, software-triggered single conversions, continuous/DMA off, 160.5-cycle sampling.
- ☐ Outputs: PA8 relay L, PA9 relay R, PB8 TEMP1 fail, PB9 TEMP2 fail, PA11 DC fault, PA12 WAIT, PB4 overheat.
- ☐ Set all protection outputs to initial LOW; use push-pull outputs.
- ☐ Inputs: PB0 mute/enable and PB1 fault reset, both pull-up.
- ☐ Do not use PA15/D2.
- ☐ Clock: HSI16 -> PLL /1 x8 /2 -> SYSCLOCK 64 MHz; AHB /1, APB /1.
- ☐ Project Manager: Advanced structure, toolchain STM32CubeIDE, Generate Under Root checked, main() generated.
- ☐ Click GENERATE CODE.

B. CubeIDE - after CubeMX has generated the project

- ☐ Open/import C:\STProjects\protection_controller\ in STM32CubeIDE.
- ☐ Add the custom headers under Core/Inc and sources under Core/Src.
- ☐ Include protection.h in main.c inside USER CODE Includes.
- ☐ Call Protection_Init() after MX_GPIO_Init() and MX_ADC1_Init().
- ☐ Call Protection_Task() repeatedly in the main loop with the 5 ms loop delay used in the project.
- ☐ Build and confirm Debug/protection_controller.elf is created.
- ☐ Connect NUCLEO-G031K8 to the PC by the onboard ST-LINK USB connector.
- ☐ Run as STM32 C/C++ Application (SWD).
- ☐ Confirm target detection, programming, and verification succeed.
- ☐ Reset/power-cycle and verify the firmware starts with both relays disconnected.

9. Troubleshooting

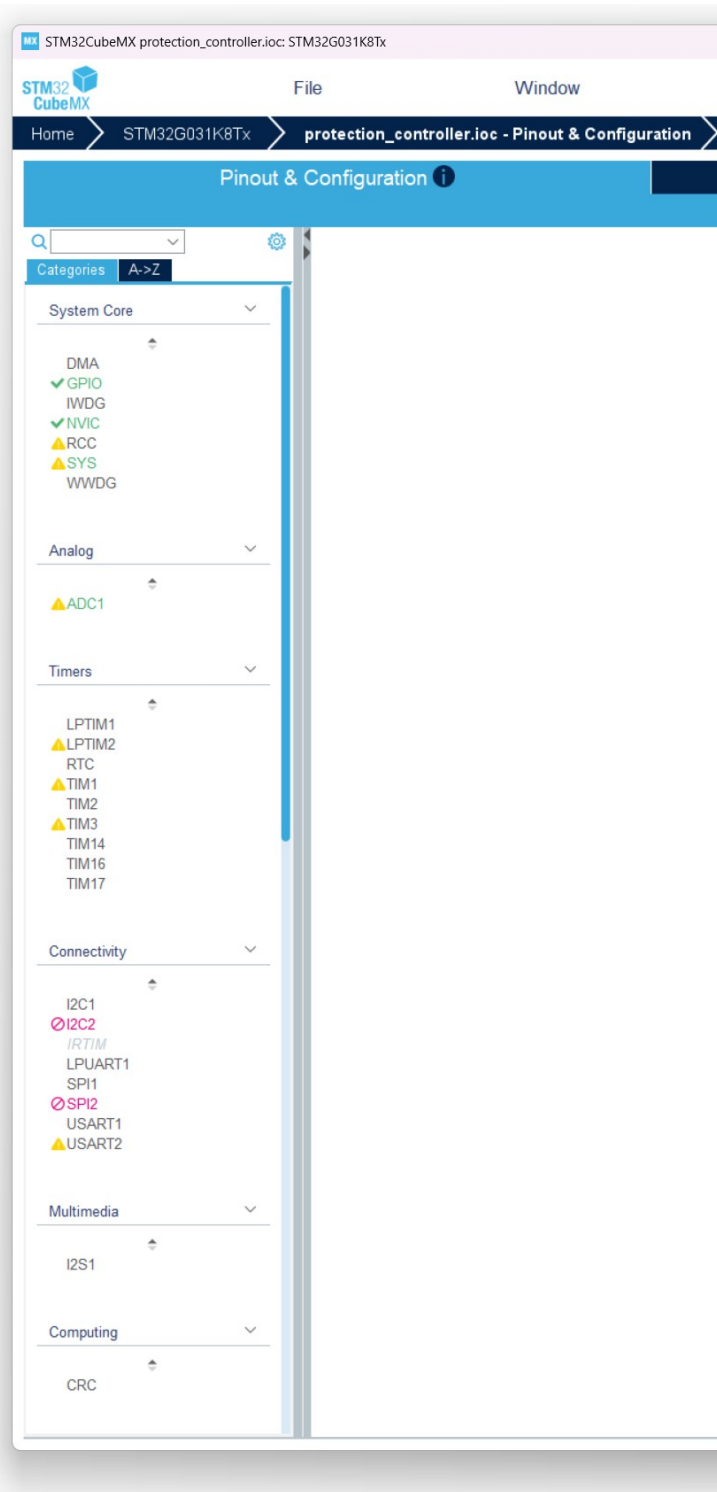
Symptom	Likely check / fix
CubeIDE cannot see the Nucleo	Use a USB data cable, reconnect the onboard ST-LINK USB connector, verify ST-LINK appears in the OS, then retry.
ST-LINK connects but cannot program	Make sure PA13/PA14 are not repurposed; set SYS Debug = Serial Wire in CubeMX and regenerate.
Firmware builds but wrong pins switch	Do not patch main.h manually. Open protection_controller.ioc, correct the CubeMX pin assignment/user labels, and regenerate.
Relay energises briefly during reset	Confirm relay GPIO initial output level is LOW in CubeMX and check the external transistor/relay driver polarity.
Buttons behave backwards	PB0/PB1 use pull-ups and are active LOW; pressed = 0.
ADC readings unstable	Verify the sensing-divider wiring/ground, 160.5-cycle sampling time, averaging, and the expected 0-3.3 V ADC range.
Temperature fault appears immediately	Check both NTC sensors and their 10 kOhm divider/pull-up networks; firmware intentionally treats invalid sensors as unsafe.
Flashing succeeds but code seems not to run	Press reset, ensure the correct Debug/protection_controller.elf was built, and check that execution did not remain halted at a debugger breakpoint.
Regeneration deletes hand edits	Keep application changes only in CubeMX USER CODE sections and in separate custom .c/.h files.
PA15/D2 does not behave as expected	Leave it unused for this design unless the Nucleo D2 jumper arrangement is intentionally changed and re-verified.

9.1 The most important recovery rule

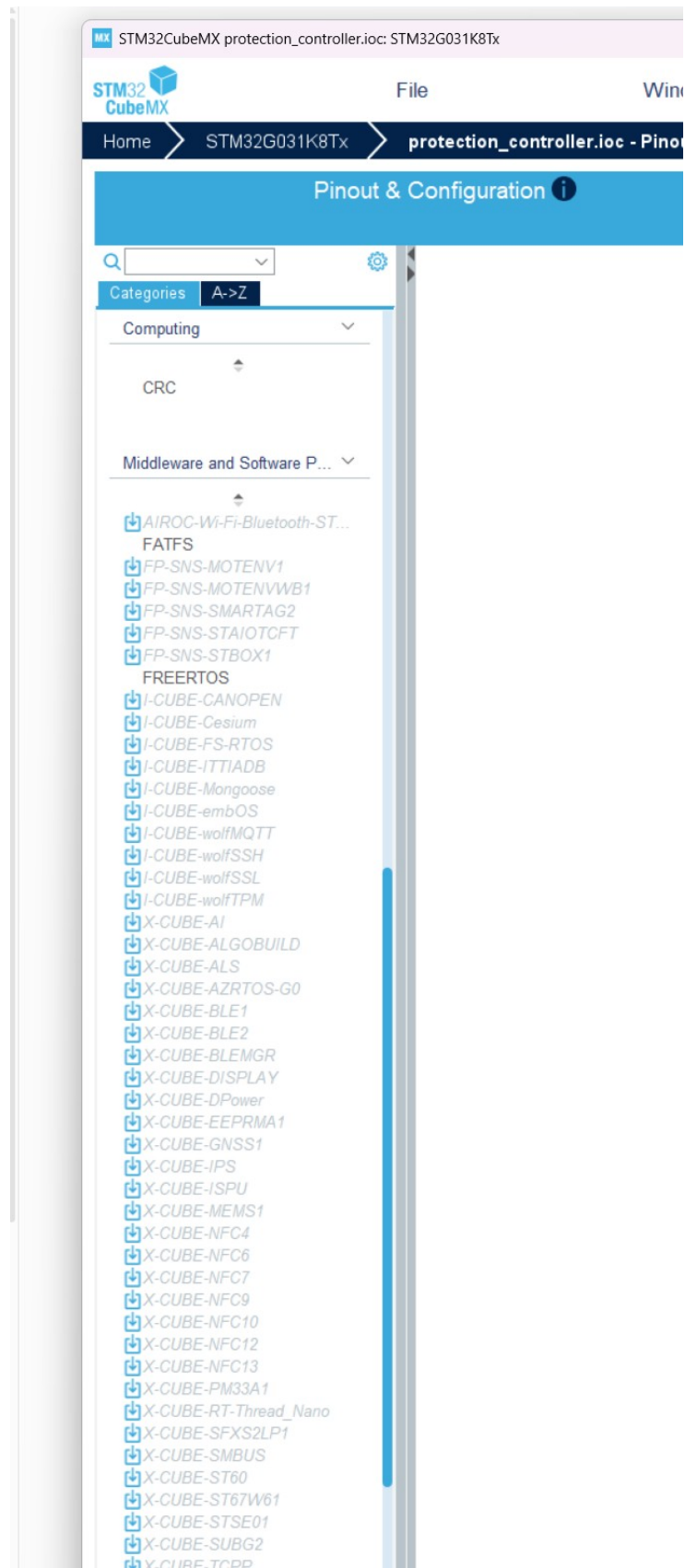
If hardware configuration is wrong: Return to protection_controller.ioc in STM32CubeMX, fix the pin/peripheral/clock setting there, save, regenerate, rebuild in CubeIDE, and flash again. This keeps the generated HAL code consistent with the hardware design.

Appendix A - Reference screenshots from the saved CubeMX project

These screenshots were supplied from the actual protection_controller.ioc project and are included as visual references for the standalone CubeMX configuration used before CubeIDE.



A1 - Pinout & Configuration categories; GPIO and ADC1 are present in the project.



A2 - Middleware list. No RTOS or middleware package was required for the protection controller.

The screenshot displays the STM32CubeMX Project Manager interface. The top bar shows the project name: STM32CubeMX: protection_controller: STM32G031K8Tx. The main window is divided into four tabs: Pinout & Configuration, Clock Configuration, Project Manager (selected), and Tools. The Project Manager tab is active, showing various settings for the project.

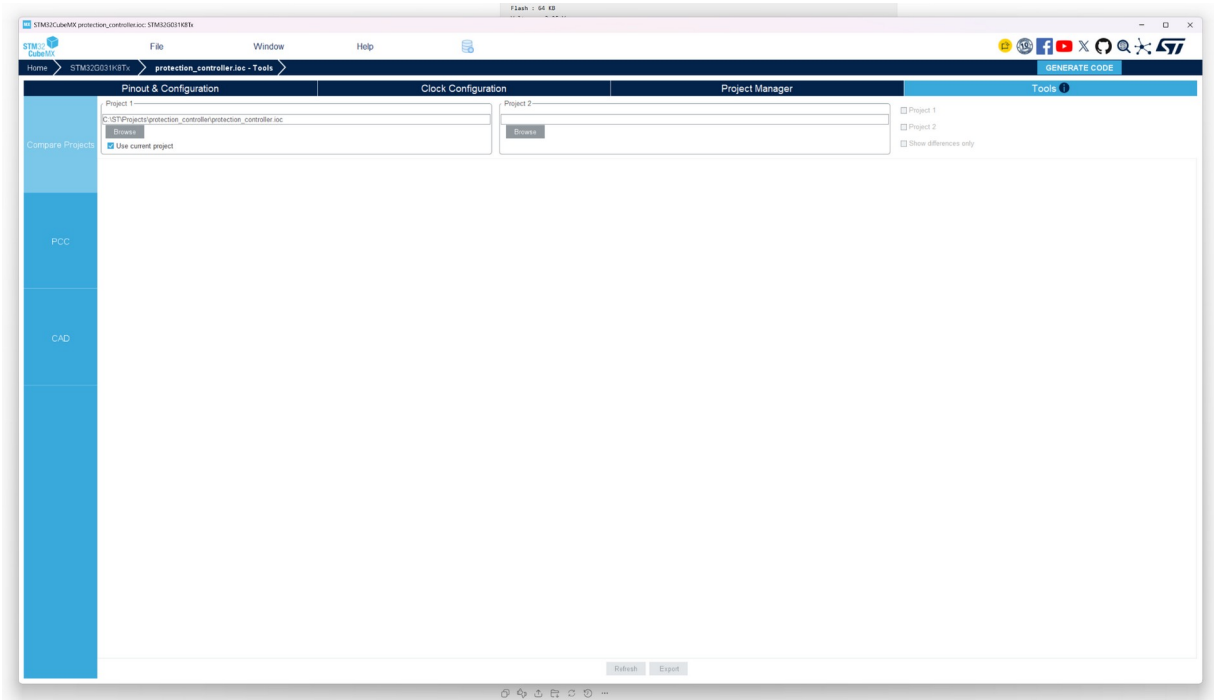
Project Settings:

- Project Name: protection_controller
- Project Location: C:\STM32CubeMX\Projects (Browse button)
- Application Structure: Advanced (Do not generate the main)
- Toolchain Folder Location: C:\STM32CubeMX\Projects\protection_controller
- Toolchain / IDE: STM32CubeIDE (Generate Under Root)

Advanced Settings:

- Linker Settings:
 - Minimum Heap Size: 6x256
 - Minimum Stack Size: 6x4096
- Thread-safe Settings:
 - Context MHAHS
 - ☐ Enable multi-threaded support
 - Thread-safe Locking Strategy: Default - Mapping suitable strategy depending on RTOS selection
- Mcu and Firmware Package:
 - Mcu Reference: STM32G031K8Tx
 - Firmware Package Name and Version: STM32Cube_FW_G0_V1.6.3 (Use latest available version)
 - ☒ Use Default Firmware Location
 - Firmware Relative Path: C:\Users\user\STM32Cube\Repository\STM32Cube_FW_G0_V1.6.3 (Browse button)

Power Amplifier Project | Page 18



A5 - CubeMX Tools page showing the current `protection_controller.ioc` project path.

Appendix B - One-page configuration cheat sheet

Category	Exact setting
Target	NUCLEO-G031K8 / STM32G031K8T6; CubeMX file target STM32G031K8Tx
ADC	PA0 IN0 TEMP1; PA1 IN1 TEMP2; PA4 IN4 DC_LEFT; PA5 IN5 DC_RIGHT; 12-bit; 160.5 cycles; software controlled
Relays	PA8 RELAY_L_CTRL; PA9 RELAY_R_CTRL; push-pull; initial LOW
LEDs	PB8 TEMP1_FAIL; PB9 TEMP2_FAIL; PA11 DC_FAULT; PA12 WAIT; PB4 OVERHEAT
Buttons	PB0 MUTE_ENABLE and PB1 FAULT_RESET; input pull-up; active LOW
Debug	SYS = Serial Wire; PA13 SWDIO; PA14 SWCLK
Avoid	PA15 / Nucleo D2 is unused in this design
Clock	HSI16 -> PLL /1 x8 /2 -> 64 MHz; AHB /1; APB /1
CubeMX project	protection_controller; C:\STProjects; Advanced; STM32CubeIDE; Generate Under Root
CubeIDE custom files	protection.*, protection_config.h, protection_io.*, protection_pins.h, sensors.*
main.c	MX_GPIO_Init(); MX_ADC1_Init(); then Protection_Init(); main loop Protection_Task(); HAL_Delay(5);
Build/flash	Build -> Debug/protection_controller.elf -> onboard ST-LINK USB -> SWD 4000 kHz -> program/verify -> reset

Remember: CubeMX defines the hardware. CubeIDE builds and flashes the generated project plus our application logic. Keeping that separation is what makes the project reproducible.